# Corrigé des exercices d'entraînement

Requêtes et explications

Master 1 Informatique médicale — TP d'initiation au SQL — Base Eunomia GiBleed (format OMOP)

## Niveau 1 — Lire une table

### Exercice 1.1 — Premier contact

```
SELECT *
FROM person
LIMIT 5;
```

**Explication**

`SELECT *` demande toutes les colonnes, `FROM person` indique la table, `LIMIT 5` évite d'afficher les 2 694 lignes.

C'est le bon réflexe pour découvrir une table inconnue : regarder quelques lignes avant d'écrire une vraie requête.

À observer : gender_concept_id contient un code (8507 = homme, 8532 = femme) tandis que gender_source_value contient la valeur d'origine (M ou F). Beaucoup de colonnes sont vides (NULL).

### Exercice 1.2 — Choisir ses colonnes

```
SELECT person_id, year_of_birth, gender_source_value
FROM person
LIMIT 10;
```

**Explication**

On remplace `*` par la liste des colonnes souhaitées, séparées par des virgules. L'ordre des colonnes dans le résultat est celui de la liste.

Sans `ORDER BY`, l'ordre des lignes n'est pas garanti : vous pouvez obtenir d'autres patients sans être en erreur.

### Exercice 1.3 — Filtrer avec WHERE

```
SELECT person_id, year_of_birth
FROM person
WHERE year_of_birth < 1920;
```

**Explication**

`WHERE` ne conserve que les lignes pour lesquelles la condition est vraie. Ici, 98 patients sont nés avant 1920.

`< 1920` exclut 1920. Pour l'inclure, il faudrait écrire `<= 1920`.

### Exercice 1.4 — Combiner des conditions et trier

```
SELECT person_id, year_of_birth
FROM person
WHERE gender_source_value = 'F'
  AND year_of_birth >= 1980
ORDER BY year_of_birth DESC;
```

**Explication**

Les deux conditions doivent être vraies en même temps : on les relie par `AND`. Avec `OR`, il suffirait que l'une des deux soit vraie.

La plus jeune est celle dont l'année de naissance est la plus grande : on trie donc par ordre décroissant (`DESC`).

`ORDER BY` se place toujours après `WHERE`. On obtient 65 patientes.

### Exercice 1.5 — Les doyens

```
SELECT person_id, year_of_birth, month_of_birth, day_of_birth
FROM person
ORDER BY year_of_birth, month_of_birth, day_of_birth
LIMIT 5;
```

#### Explication

Les plus âgés sont nés le plus tôt : on trie par ordre croissant (c'est l'ordre par défaut, sans DESC).

Trier sur trois colonnes permet de départager les patients nés la même année : on compare d'abord l'année, puis le mois, puis le jour.

`LIMIT 5` garde les 5 premiers du classement. LIMIT vient toujours en dernier.

### Exercice 1.6 — Valeurs distinctes

```
SELECT DISTINCT domain_id
FROM concept;
```

#### Explication

Sans `DISTINCT`, on obtiendrait 444 lignes (une par concept) avec beaucoup de répétitions. `DISTINCT` ne garde qu'un exemplaire de chaque valeur : on obtient 8 domaines.

Dans OMOP, le domaine d'un concept détermine la table où il est rangé : un concept du domaine Condition se trouve dans condition_occurrence, un concept du domaine Drug dans drug_exposure, etc.

### Exercice 1.7 — Chercher dans le dictionnaire

```
SELECT concept_id, concept_name, domain_id
FROM concept
WHERE concept_name LIKE '%Ibuprofen%';
```

#### Explication

`LIKE` compare un texte à un modèle. `%` signifie « zéro, un ou plusieurs caractères quelconques » : `'%Ibuprofen%'` trouve donc le mot au début, au milieu ou à la fin du nom.

On obtient 3 concepts : la substance Ibuprofen et deux médicaments (comprimés à 100 mg et 200 mg). C'est ainsi qu'on retrouve le code d'un médicament ou d'une maladie avant de l'utiliser dans une autre requête.

Dans SQLite, LIKE ne tient pas compte des majuscules : `'%ibuprofen%'` donne le même résultat.

## Niveau 2 — Compter et résumer

### Exercice 2.1 — Compter des lignes

```
SELECT COUNT(*) FROM condition_occurrence;

SELECT COUNT(*) FROM person;
```

**Explication**

`COUNT(*)` compte toutes les lignes de la table et renvoie un seul nombre.

La base contient 65 332 diagnostics pour 2 694 patients : un patient a donc en moyenne environ 24 diagnostics.

### Exercice 2.2 — Clé primaire et doublons

```
SELECT COUNT(*) AS nb_lignes, COUNT(DISTINCT person_id) AS nb_patients
FROM person;

SELECT COUNT(*) AS nb_lignes, COUNT(DISTINCT person_id) AS nb_patients
FROM condition_occurrence;
```

**Explication**

Dans person, les deux nombres sont égaux (2 694) : chaque person_id n'apparaît qu'une fois. C'est cohérent avec son rôle de clé primaire.

Dans condition_occurrence, il y a 65 332 lignes mais seulement 2 694 person_id différents : un même patient apparaît sur plusieurs lignes. Ici person_id n'est pas une clé primaire mais une clé étrangère : elle fait référence au patient de la table person. La clé primaire de condition_occurrence est condition_occurrence_id.

À retenir : COUNT(*) compte des lignes (des événements), COUNT(DISTINCT person_id) compte des patients.

### Exercice 2.3 — Les valeurs manquantes

```
SELECT COUNT(*) AS nb_sans_date_fin
FROM condition_occurrence
WHERE condition_end_date IS NULL;
```

**Explication**

8 652 diagnostics n'ont pas de date de fin : il s'agit en général de maladies chroniques, toujours en cours.

NULL signifie « valeur absente ou inconnue ». Ce n'est ni zéro, ni un texte vide. On ne peut pas écrire `= NULL` : la comparaison ne renvoie jamais vrai, et la requête donne 0. Il faut écrire `IS NULL` (ou `IS NOT NULL` pour l'inverse).

### Exercice 2.4 — Statistiques simples

```
SELECT MIN(year_of_birth) AS plus_ancienne,
       MAX(year_of_birth) AS plus_recente,
       ROUND(AVG(year_of_birth), 0) AS moyenne
FROM person;
```

**Explication**

Les trois fonctions résument toute la colonne en une seule valeur : on obtient une seule ligne.

`ROUND(x, 0)` arrondit à 0 décimale. Les fonctions peuvent s'emboîter : on calcule d'abord la moyenne, puis on l'arrondit.

SQLite affiche 1958.0 : c'est normal, ROUND renvoie un nombre décimal.

### Exercice 2.5 — Regrouper avec GROUP BY

```
SELECT domain_id, COUNT(*) AS nb_concepts
FROM concept
GROUP BY domain_id
ORDER BY nb_concepts DESC;
```

#### Explication

`GROUP BY domain_id` forme 8 groupes (les 8 domaines vus à l'exercice 1.6) ; `COUNT(*)` compte les lignes de chaque groupe.

On peut trier sur le nom donné avec `AS` (nb_concepts). `DESC` met le plus grand groupe en premier : les médicaments (Drug) sont les plus nombreux.

Règle importante : toute colonne du SELECT qui n'est pas dans une fonction (COUNT, AVG…) doit figurer dans le GROUP BY.

### Exercice 2.6 — Les patients les plus suivis

```
SELECT person_id, COUNT(*) AS nb_diagnostics
FROM condition_occurrence
GROUP BY person_id
ORDER BY nb_diagnostics DESC
LIMIT 5;
```

#### Explication

Dans condition_occurrence, un patient apparaît sur autant de lignes qu'il a de diagnostics. Regrouper par person_id puis compter donne le nombre de diagnostics par patient.

On trie du plus grand au plus petit et on garde les 5 premiers. Cette requête combine presque toutes les clauses vues jusqu'ici, dans l'ordre obligatoire : SELECT, FROM, GROUP BY, ORDER BY, LIMIT.

### Exercice 2.7 — Filtrer des groupes avec HAVING

```
SELECT person_id, COUNT(*) AS nb_diagnostics
FROM condition_occurrence
GROUP BY person_id
HAVING COUNT(*) >= 50
ORDER BY nb_diagnostics DESC;
```

#### Explication

Le nombre de diagnostics n'existe qu'après le regroupement : on ne peut donc pas le filtrer avec WHERE. HAVING sert précisément à filtrer sur un résultat de COUNT, AVG, etc.

14 patients ont au moins 50 diagnostics.

## Niveau 3 — Relier les tables : les jointures

### Exercice 3.1 — Première jointure

```
SELECT p.person_id, p.year_of_birth, c.concept_name AS genre
FROM person p
JOIN concept c ON c.concept_id = p.gender_concept_id
LIMIT 10;
```

**Explication**

Pour chaque patient, SQL prend son gender_concept_id (8507 ou 8532) et va chercher dans concept la ligne qui a ce concept_id. Il récupère alors le concept_name correspondant (MALE ou FEMALE).

La condition après `ON` relie une clé étrangère (p.gender_concept_id) à une clé primaire (c.concept_id).

Les alias (p, c) indiquent de quelle table vient chaque colonne. Ici concept_name n'existe que dans concept, mais il est plus clair de toujours préfixer.

### Exercice 3.2 — Les diagnostics les plus fréquents

```
SELECT c.concept_name AS diagnostic, COUNT(*) AS nb
FROM condition_occurrence co
JOIN concept c ON c.concept_id = co.condition_concept_id
GROUP BY c.concept_name
ORDER BY nb DESC
LIMIT 10;
```

**Explication**

La jointure ajoute à chaque diagnostic son libellé, puis on regroupe par libellé et on compte : c'est la combinaison des niveaux 2 et 3.

Synthea simule une population générale : les pathologies bénignes et courantes dominent (sinusites, pharyngites, bronchites).

### Exercice 3.3 — Les médicaments les plus prescrits

```
SELECT c.concept_name AS medicament, COUNT(*) AS nb
FROM drug_exposure d
JOIN concept c ON c.concept_id = d.drug_concept_id
GROUP BY c.concept_name
ORDER BY nb DESC
LIMIT 10;
```

**Explication**

Même structure que l'exercice précédent : seuls la table de départ et la colonne de jointure changent. Savoir réutiliser une requête existante est une compétence essentielle.

On y trouve des vaccins : dans OMOP, ils font partie du domaine Drug.

### Exercice 3.4 — Le dossier d'un patient

```
SELECT co.condition_start_date, c.concept_name AS diagnostic
FROM condition_occurrence co
JOIN concept c ON c.concept_id = co.condition_concept_id
WHERE co.person_id = 6
ORDER BY co.condition_start_date;
```

**Explication**

On combine jointure, filtre et tri : on obtient l'histoire médicale de la patiente n° 6 (17 diagnostics, de l'otite de l'enfance à l'arthrose).

C'est exactement ce que fait un logiciel de dossier patient quand il affiche les antécédents : une requête de ce type, avec le numéro du patient en paramètre.

### Exercice 3.5 — Lignes ou patients ?

```
SELECT COUNT(*) AS nb_diagnostics,
       COUNT(DISTINCT person_id) AS nb_patients
FROM condition_occurrence
WHERE condition_concept_id = 192671;
```

**Explication**

Pas de jointure nécessaire ici : on connaît déjà le code du diagnostic.

Les deux nombres sont égaux (479) : chaque patient concerné n'a qu'un seul épisode d'hémorragie dans la base. Ce n'est pas toujours le cas : pour la sinusite virale, il y a 17 268 diagnostics pour 2 686 patients.

En épidémiologie, cette distinction est fondamentale : un nombre d'événements n'est pas un nombre de personnes.

## Niveau 4 — Questions cliniques

### Exercice 4.1 — Âge au moment de l'hémorragie

```
SELECT ROUND(AVG(strftime('%Y', co.condition_start_date)
                 - p.year_of_birth), 1) AS age_moyen,
       MIN(strftime('%Y', co.condition_start_date) - p.year_of_birth) AS age_min,
       MAX(strftime('%Y', co.condition_start_date) - p.year_of_birth) AS age_max
FROM condition_occurrence co
JOIN person p ON p.person_id = co.person_id
WHERE co.condition_concept_id = 192671;
```

#### Explication

Première jointure entre deux tables de données (et non avec le dictionnaire) : condition_occurrence.person_id est une clé étrangère vers person.person_id.

Pour chaque diagnostic, on calcule l'âge, puis AVG, MIN et MAX résument ces âges. Les patients ont entre 32 et 47 ans, 38,9 ans en moyenne.

strftime renvoie l'année sous forme de texte ; SQLite la convertit automatiquement en nombre pour faire la soustraction.

### Exercice 4.2 — Répartition par genre

```
SELECT c.concept_name AS genre, COUNT(DISTINCT co.person_id) AS nb_patients
FROM condition_occurrence co
JOIN person p  ON p.person_id = co.person_id
JOIN concept c ON c.concept_id = p.gender_concept_id
WHERE co.condition_concept_id = 192671
GROUP BY c.concept_name;
```

#### Explication

On enchaîne deux jointures : chaque JOIN ajoute une table et sa condition de liaison.

Résultat : 242 femmes et 237 hommes. La répartition est presque la même que dans l'ensemble de la base (1 373 femmes, 1 321 hommes) : dans ces données, le genre ne semble pas lié au risque d'hémorragie.

Comparer à la population de référence est un réflexe à acquérir : un chiffre isolé ne dit rien.

### Exercice 4.3 — Exposition aux anti-inflammatoires

```
SELECT COUNT(DISTINCT co.person_id) AS nb_patients
FROM condition_occurrence co
JOIN drug_exposure d ON d.person_id = co.person_id
WHERE co.condition_concept_id = 192671
  AND d.drug_concept_id IN (1118084, 1124300);
```

#### Explication

La jointure relie, pour un même patient, ses diagnostics et ses prescriptions.

Résultat : 479, c'est-à-dire tous les patients ayant eu une hémorragie. C'est surprenant… et c'est voulu : la base a été construite pour une étude comparant célécoxib et diclofénac, et chaque patient a reçu l'un des deux.

Message important : un résultat doit toujours être confronté à ce que l'on sait des données. Une requête juste peut donner un résultat qui s'explique par la façon dont la base a été constituée.

### Exercice 4.4 — Exposition avant l'événement

```
SELECT c.concept_name AS medicament,
       COUNT(DISTINCT co.person_id) AS nb_patients
FROM condition_occurrence co
JOIN drug_exposure d ON d.person_id = co.person_id
JOIN concept c       ON c.concept_id = d.drug_concept_id
WHERE co.condition_concept_id = 192671
  AND d.drug_concept_id IN (1118084, 1124300)
  AND d.drug_exposure_start_date < co.condition_start_date
GROUP BY c.concept_name;
```

#### Explication

C'est la logique d'une étude de pharmaco-épidémiologie : pour qu'un médicament puisse être mis en cause, l'exposition doit précéder l'événement.

La comparaison de dates fonctionne parce qu'elles sont au format AAAA-MM-JJ : l'ordre alphabétique et l'ordre chronologique coïncident.

355 patients avaient commencé le célécoxib avant l'hémorragie, 124 le diclofénac. Ces chiffres bruts ne permettent pas de conclure que l'un est plus risqué que l'autre : il faudrait les rapporter au nombre de patients exposés (voir le bonus 5.5).

## Niveau 5 — Bonus : pour aller plus loin

### Exercice 5.1 — Patients par décennie de naissance

```
SELECT (year_of_birth / 10) * 10 AS decennie,
       COUNT(*) AS nb_patients
FROM person
GROUP BY decennie
ORDER BY decennie;
```

**Explication**

On regroupe sur une valeur calculée : (1963 / 10) × 10 = 196 × 10 = 1960.

Ce comportement de la division entière dépend du logiciel : d'autres bases renverraient 196,3.

### Exercice 5.2 — Code source et code standard

```
SELECT std.concept_name AS libelle_standard, std.vocabulary_id AS voc_standard,
       src.concept_name AS libelle_source,   src.vocabulary_id AS voc_source,
       COUNT(*) AS nb
FROM condition_occurrence co
JOIN concept std ON std.concept_id = co.condition_concept_id
JOIN concept src ON src.concept_id = co.condition_source_concept_id
WHERE std.concept_name LIKE '%hemorrhage%'
GROUP BY 1, 2, 3, 4;
```

**Explication**

Une même table peut être jointe plusieurs fois, à condition de lui donner un alias différent à chaque fois.

On voit le principe d'OMOP : le code d'origine (CIM-10-CM K92.2) est « traduit » vers un concept standard SNOMED. Cela permet de comparer des bases qui utilisaient au départ des codages différents.

`GROUP BY 1, 2, 3, 4` désigne les colonnes par leur position dans le SELECT.

### Exercice 5.3 — Durée de suivi

```
SELECT ROUND(AVG(julianday(observation_period_end_date)
                 - julianday(observation_period_start_date)) / 365.25, 1)
       AS duree_moyenne_annees
FROM observation_period;
```

**Explication**

On ne peut pas soustraire directement deux dates texte : julianday les convertit en nombres de jours.

Notion importante dans OMOP : un patient n'est « observable » que pendant ses périodes d'observation. En dehors, l'absence d'information ne veut pas dire absence de maladie.

### Exercice 5.4 — Utiliser la hiérarchie des concepts

```
SELECT c.concept_name AS produit, COUNT(*) AS nb
FROM drug_exposure d
JOIN concept_ancestor ca ON ca.descendant_concept_id = d.drug_concept_id
JOIN concept c          ON c.concept_id = d.drug_concept_id
WHERE ca.ancestor_concept_id = 1177480
GROUP BY c.concept_name;
```

**Explication**

C'est l'usage principal de concept_ancestor dans les études OMOP : définir une exposition au niveau de la substance active, quels que soient le dosage et la forme du médicament.

Dans une vraie base, une substance peut correspondre à des centaines de produits : la hiérarchie évite de les lister à la main.

### Exercice 5.5 — Comparer exposés et non exposés

```
WITH exposes AS (
    SELECT DISTINCT person_id FROM drug_exposure
    WHERE drug_concept_id = 1118084
),
hemorragie AS (
    SELECT DISTINCT person_id FROM condition_occurrence
    WHERE condition_concept_id = 192671
)
SELECT CASE WHEN e.person_id IS NOT NULL THEN 'Exposé'
            ELSE 'Non exposé' END AS groupe,
       COUNT(*) AS nb_patients,
       COUNT(h.person_id) AS nb_hemorragie,
       ROUND(100.0 * COUNT(h.person_id) / COUNT(*), 1) AS pourcentage
FROM person p
LEFT JOIN exposes e    ON e.person_id = p.person_id
LEFT JOIN hemorragie h ON h.person_id = p.person_id
GROUP BY groupe;
```

#### Explication

WITH crée deux listes temporaires de patients, que l'on utilise ensuite comme des tables.

LEFT JOIN conserve tous les patients, même ceux absents des listes (leurs colonnes valent alors NULL). COUNT(h.person_id) ne compte pas les NULL.

On écrit 100.0 et non 100 : sinon la division serait entière et donnerait 0.

Résultat : 19,3 % d'hémorragies chez les exposés au célécoxib, 14,6 % chez les autres (qui, dans cette base, sont tous sous diclofénac). Comparaison brute, non ajustée : elle ne prouve pas que le célécoxib est plus risqué (facteurs de confusion, données synthétiques).